LINEAR REGRESSION — Boston Housing Dataset

What is Linear Regression

Linear regression is one of the most fundamental algorithms in machine learning. The core idea is to find a line or surface that best fits the data so we can predict a continuous output like house prices from a set of input features. The model learns by minimizing the gap between what it predicts and the actual answer. These gaps are called residuals or errors. The internal values the model adjusts are called weights or coefficients.

I built this from scratch using gradient descent. Gradient descent works by figuring out which direction reduces the error the most at any point during training, then taking a small step in that direction. This keeps repeating until the error stops getting smaller, which is called convergence. The size of each step is controlled by the learning rate. If the learning rate is too high the model overshoots and the loss bounces around or blows up. If it is too low training becomes painfully slow. A loss function measures the error for a single data point while a cost function is the average of all those individual losses over the entire training set. During training we always minimize the cost function. We also divide the gradient by the number of samples to keep the gradient scale stable regardless of dataset size, so the same learning rate works whether we have 100 or 100,000 samples.

Understanding the Data through EDA

Before building any model I spent time understanding the dataset through exploratory data analysis. The Boston Housing dataset has 506 rows and 13 features. The goal was to predict MEDV, the median house value in thousands of dollars.

Looking at the target variable distribution plot, the mean price is 22.5k and the median is 21.2k. The distribution is slightly right skewed meaning most homes are moderately priced with fewer expensive ones. There is a spike at 50k caused by a data collection cap, which the Q-Q plot confirms through deviations at the upper tail. The boxplot also shows several high value outliers above 35k.

The feature distributions plot shows that most features are far from normally distributed. CRIM has a skew of 5.22 and ZN has 2.23, both heavily bunched near zero with a few extreme outliers. CHAS is almost entirely zero since very few neighborhoods border the river. AGE is left skewed meaning most properties are old. RM is the most normally shaped feature with a skew of just 0.40. These skewness patterns tell us which features might need transformations before modelling.

Correlation and Multicollinearity

The correlation heatmap revealed the most important relationships in the data. RM had the strongest positive correlation with MEDV at 0.70 and LSTAT had the strongest negative at 0.74. NOX and INDUS were strongly linked at 0.76. But the most alarming number in the heatmap

was the 0.91 correlation between RAD and TAX, which flagged a serious multicollinearity problem.

Multicollinearity happens when two or more features carry almost the same information. The model gets confused about which one is actually influencing the prediction and the coefficient estimates become unstable and unreliable. Even small changes in data can flip the sign of a coefficient when multicollinearity is severe. It does not always hurt prediction accuracy but it makes the model almost impossible to interpret correctly.

To confirm this I calculated the Variance Inflation Factor for every feature. VIF measures how much the variance of a coefficient is inflated because of its correlation with other features. The VIF chart shows TAX scored 9.01 and RAD scored 7.48, both highlighted in orange and crossing the moderate threshold of 5. Every other feature stayed comfortably below 5. This confirmed TAX and RAD are essentially duplicating information since highway access and property tax rates tend to move together across neighborhoods.

Assumptions of Linear Regression

Linear regression relies on several assumptions for its results to be trustworthy. The relationship between features and the target must be linear. The errors must be independent of each other, especially important in time series data. The variance of errors must stay constant across all prediction levels, which is called homoscedasticity. If the spread of errors grows with predicted values that is called heteroscedasticity and it makes confidence intervals unreliable. The residuals should be approximately normally distributed for valid statistical inference. There must be no severe multicollinearity among features. And the features must not be correlated with the error term, a condition called exogeneity. Violating exogeneity leads to biased coefficients, which is the most serious kind of violation.

Training and Results

Looking at the gradient descent convergence plot, the MSE loss started around 300 and dropped sharply in the first 100 iterations before flattening out after around 200 iterations. The zoomed in version on the right confirms the rapid early learning. This from scratch model achieved a test R squared of 0.6386 and a test RMSE of 5.1482, which was very close to the sklearn result and confirmed the implementation was correct.

The standardized coefficients plot shows RM with the largest positive effect at 3.32, meaning more rooms strongly pushes prices up after accounting for all other features. LSTAT had the most negative coefficient at 3.54, confirming it is the biggest drag on prices. DIS (distance to employment) was 3.01 negative and NOX (pollution) was 1.80 negative. CHAS had a positive effect of 0.79 since proximity to the river adds value.

The sklearn baseline achieved a train R squared of 0.7433 and a test R squared of 0.6390 with a test RMSE of 5.145, meaning the model was on average about 5 thousand dollars off on unseen data. The 5 fold cross validation R squared was 0.7001 with a standard deviation of 0.0448, which is a reliable generalization estimate.

R squared measures what proportion of variance in house prices the model explains. However R squared always increases when you add more features even useless ones. Adjusted R squared fixes this by penalizing extra features and only goes up if a new feature genuinely contributes. In our case the adjusted R squared on test data dropped to 0.5904, which is more honest than the raw 0.6390. The gap between training R squared (0.7433) and test R squared (0.6390) revealed mild overfitting, which is what motivated using regularization.

We use MSE as the loss function for linear regression because it is smooth and differentiable everywhere, which makes gradient descent work cleanly. MSE squares each error so large mistakes get penalized much more than small ones, pushing the model to reduce big errors first. MAE treats all errors equally and is not differentiable at zero, creating problems for optimization. Huber loss is a hybrid that behaves like MSE for small errors and like MAE for large ones, making it more robust to outliers while remaining differentiable.

Regularization

Regularization addresses overfitting by adding a penalty to the cost function for large coefficients, forcing the model to stay simpler and generalize better. The strength of regularization is controlled by a parameter called lambda or alpha. A very high penalty leads to underfitting. A very small penalty does not control overfitting.

Lasso (L1) adds the sum of absolute coefficient values as a penalty. Its unique property is that it can push some coefficients all the way to exactly zero, effectively removing those features from the model entirely, which is called automatic feature selection. The Lasso coefficient path plot shows features disappearing one by one as the penalty increases. The MSE cross validation chart on the right found the best alpha at 0.0594. At that point Lasso dropped 2 features entirely and achieved a test R squared of 0.6387.

Ridge (L2) adds the sum of squared coefficient values. It never zeroes out a coefficient but shrinks them all toward zero. This makes Ridge better when all features are somewhat meaningful but correlated, since it specifically handles multicollinearity. Ridge achieved the best test R squared among all models at 0.6435 with the smallest overfit gap of 0.0973.

ElasticNet combines both penalties to get the feature selection property of Lasso and the stability of Ridge. It achieved a test R squared of 0.6416.

The model performance comparison chart shows all four models side by side on R squared, RMSE, and coefficient values. All four performed very similarly on this dataset. The coefficient comparison panel on the right clearly shows how Lasso zeroes some features while Ridge smoothly shrinks them all. The RMSE stayed around 5.1k across all models meaning predictions were consistently off by about 5 thousand dollars on average.

Residual Analysis

The residual plots are essential for checking whether model assumptions hold. The actual vs predicted chart shows points following the diagonal reasonably well but with more scatter at

high price values. The residuals vs predicted chart shows errors mostly random around zero in the middle range but fanning out slightly at higher predicted values, which is a mild sign of heteroscedasticity. The residual distribution is roughly bell shaped. The Q-Q plot follows the normal line well in the centre but deviates at the tails, partly because of the 50k price cap creating unusual behavior at the extremes.

Summary

The model explains roughly 70% of variance in Boston house prices, which is solid for a linear model. Number of rooms and lower status population percentage are the two most dominant drivers of price. TAX and RAD showed severe multicollinearity at VIF scores of 9.01 and 7.48. Ridge regularization handled this best and gave the highest test performance. Building gradient descent from scratch made the optimization process intuitive and confirmed a genuine understanding of how the model learns.